Dr. Ekta Backend Documentation

Dr. Ekta Narula Dental Care — Backend Documentation
1. Project Overview
2. Quick Start
3. Project Structure
4. Architecture & Request Flow
5. Authentication & Authorization
6. Database Models / Schema
   User (admins)
   RefreshToken
   ClinicInfo (singleton, `singleton: "clinic_info"` unique)
   MediaAsset (every uploaded file)
   ContentReview — REMOVED (review)
   Treatment (treatment pages, draft/published)
   Article (knowledge-centre posts, draft/published)
   Lead (CRM rows from forms and clicks)
   BookingProviderConfig (singleton, `singleton: "booking_provider"` unique)
   Profile (doctor singleton — no approval workflow)
   Testimonial (patient stories, draft/published)
   GalleryItem (before/after pairs, draft/published)
   Video (YouTube embeds — plain CRUD, no approval workflow)
   CuratedPost (manual Instagram grid — plain CRUD)
   Announcement (clinic notices, draft/published)
   HomepageConfig (singleton, no `singleton` key — first document wins)
   RedirectMap
   AnalyticsEvent (server-side event log)
   Location (no API endpoints — schema only)
   AuditLog — REMOVED (review)
7. Complete API Reference
   7.1 Root, health, clinic info (public)
   7.2 Treatments & articles (public)
   7.3 Profile, homepage, testimonials, gallery, videos, Instagram (public)
   7.4 Enquiry, booking CTA, click log (public writes)
   7.5 SEO (public)
   7.6 Admin auth (no token needed)
   7.7 Admin clinic-info and media
   7.8 Admin treatments + articles (draft/published, no workflow)
   7.9 Admin testimonials + gallery (draft/published, no consent gate)
   7.10 Admin videos, Instagram posts, homepage config (plain CRUD)
   7.11 Admin leads
   7.12 Admin booking provider, redirects, analytics
8. Function & Module Reference
   `src/app.js` / `src/config/*` / `src/utils/*` / `infra`
   Middlewares

Models (statics, methods, hooks)
Services
Controllers (handler → route)
9. Standard Conventions
10. Frontend Integration Guide
11. Known Gaps & Warnings
12. One-Day Integration Checklist

Dr. Ekta Narula Dental Care — Backend Documentation

> Handover for the frontend engineer. Covers the **current** code (2FA has been removed). **Review update (2026-09-21, branch `review/muse-improvements`):** single-admin simplifications landed — audit logging removed, patient-consent gate removed, content workflow collapsed to `draft`/`published`, XSS sanitizing wired on rich-text writes, public payloads exclude internal fields, CSV export streams with formula escaping, sitemap/SEO bugs fixed, route-shadowing fixed, `/auth/me` protected. Sections below marked ~~strikethrough~~ where behavior was deleted; "FIXED" where a flagged bug is resolved. Base URL (local): `http://localhost:4000`. API prefix: `/api`.

---

1. Project Overview

- This backend is the API for the **Dr. Ekta Narula Dental Care** clinic website.
- It serves all public website content: treatments, articles, videos, gallery, testimonials, doctor profile, clinic info, and homepage content.
- It collects patient enquiries through a public form and stores them as **leads** for the clinic team.
- It provides a full **admin API** (single admin) to create, publish, and delete content, manage leads, upload media, manage redirects, and view analytics. There is no review workflow and no audit trail (both removed by owner decision).
- Content is either `draft` or `published`. Only `published` items are visible on public endpoints. The admin sets `status` directly on create/update.
- Media files are uploaded **directly from the browser to Cloudinary** using a signed URL from the backend. The backend only stores file metadata.
- There is **no signup, no patient login, no cart, no payments, no websockets**. The only login is the single admin login (email + password, JWT).
- External booking is a dumb link system: the backend stores the Practo profile/booking URLs and the frontend renders a "Book" button pointing at them.

Tech stack

Layer	Technology
Language	JavaScript (Node.js)
Framework	Express 4
Database	MongoDB + Mongoose 8

Layer	Technology
Auth	JWT access + rotating refresh tokens (httpOnly cookie + body), bcrypt password hashing
File storage	Cloudinary (direct browser upload via signed params)
Validation	express-validator
Security	helmet, cors, express-mongo-sanitize, express-rate-limit, key sanitizer
Logging	winston
Tests	jest + supertest + mongodb-memory-server
External services	Practo (URL links only, no API), Google Analytics (config only, no server code), SMTP (config only, no sending code)

2. Quick Start

Prerequisites

- Node.js 18+ and npm
- A MongoDB instance (local `mongodb://localhost:27017/dr_ekta_db` or Atlas)
- A Cloudinary account (only needed for media upload; rest of the API works without it)

Install and run

```
cd backend
npm install
cp .env.example .env   # then fill in values (see table below)
npm run dev            # local dev with nodemon (port 4000)
# or
npm start              # production start: node src/app.js
npm test               # integration tests (uses in-memory Mongo, no real DB
         needed)
```

- Base URL local: `http://localhost:4000`
- Public API prefix: `http://localhost:4000/api/public`
- Admin API prefix: `http://localhost:4000/api/admin`
- Health check: `GET http://localhost:4000/api/public/health`
- Root info: `GET http://localhost:4000/` returns `{ name, version, status }`

Environment variables

Name	Purpose	Required?	Exa
`PORT`	Port the server listens on	No (default `4000`)	`4000`

Name	Purpose	Required?	Exa
NODE_ENV	development, production, or test	No (default development)	development
MONGODB_URI	MongoDB connection string	**Yes**	mongodb://lo
JWT_ACCESS_SECRET	Secret to sign short-lived access tokens	**Yes**	<random-long
JWT_REFRESH_SECRET	Secret to sign refresh tokens (use a different value)	**Yes**	<other-rando
JWT_ACCESS_EXPIRY	Access token lifetime	No (default 15m)	15m
JWT_REFRESH_EXPIRY	Refresh token lifetime (informational; DB expiry is hardcoded 7 days)	No (default 7d)	7d
RATE_LIMIT_WINDOW_MS	Default rate-limit window	No (default 900000)	900000
RATE_LIMIT_MAX_REQUESTS	Default max requests per window	No (default 100)	100
CLOUDINARY_CLOUD_NAME	Cloudinary account name (media upload)	Yes for uploads	<your-cloud-
CLOUDINARY_API_KEY	Cloudinary API key	Yes for uploads	<your-api-ke
CLOUDINARY_API_SECRET	Cloudinary API secret	Yes for uploads	<your-api-se
PRACTO_PROFILE_URL	Practo profile link	No (booking URLs come from DB, this env value is currently unused)	https://www.
PRACTO_BOOKING_URL	Practo booking link	No (same as above, currently unused)	https://www.
GA_MEASUREMENT_ID	Google Analytics ID	No (no server code reads it yet)	G-XXXXXXX
GA_API_SECRET	Google Analytics API secret	No (same as above)	<ga-secret>
SMTP_HOST	SMTP host for emails	No (no email-sending code exists yet)	smtp.example
SMTP_PORT	SMTP port	No (default 587)	587

Name	Purpose	Required?	Exa
SMTP_USER	SMTP username	No	<smtp-user>
SMTP_PASS	SMTP password	No	<smtp-passwo
NOTIFICATION_EMAIL	Where enquiry notifications would go	No (notifications are only a redacted console.log today)	clinic@examp
RECAPTCHA_SECRET	reCAPTCHA server-side verification for enquiries	No (when empty, only the documented "invalid-token" stub is rejected)	<recaptcha-s
FRONTEND_URL	CORS allowlist (comma-separated) + base URL used in sitemap/robots	No (default http://localhost:3000)	http://local
LOG_LEVEL	Winston log level (read directly, not in .env.example)	No (default info)	info

Notes: - The app throws on startup if MONGODB_URI, JWT_ACCESS_SECRET, or JWT_REFRESH_SECRET is missing (skipped when NODE_ENV=test). - The checked-in .env also contains STORAGE_PROVIDER and AWS_* keys — these are **dead leftovers**, the code hardcodes the Cloudinary provider and never reads them.

---

3. Project Structure

```
backend/
├── src/
│   ├── app.js                # Express app: middleware, route mounting,
startup
│   ├── config/
│   │   ├── db.js             # MongoDB connect/disconnect (pool 10–20)
│   │   ├── env.js            # Reads + validates env vars, exports config
│   │   └── integrations.config.js  # Re-exports storage/practo/GA config
(only Cloudinary is used)
│   ├── routes/               # URL → controller mapping (see Section 7)
│   │   ├── admin.routes.js       # Auth (login/refresh/logout/me), clinic-
info, media
│   │   ├── content.routes.js     # Admin treatments + articles (no
workflow routes)
│   │   ├── lead.routes.js        # Admin lead CRUD/stats/export
│   │   ├── practo.routes.js      # Public booking-cta + click log; admin
```

```
provider config
│   │   ├── profile.routes.js       # Split publicRouter + adminRouter
(doctor profile)
│   │   ├── testimonial.routes.js   # Split publicRouter + adminRouter (plain
CRUD)
│   │   ├── gallery.routes.js       # Split publicRouter + adminRouter (plain
CRUD)
│   │   ├── video.routes.js         # Separate publicRouter + adminRouter
│   │   ├── curatedPost.routes.js   # Public list + admin CRUD (Instagram
posts)
│   │   ├── announcement.routes.js    # Split publicRouter + adminRouter
(notices)
│   │   ├── homepage.routes.js      # Public aggregate + admin featured
config
│   │   ├── redirect.routes.js      # Admin redirect CRUD
│   │   ├── analytics.routes.js     # Admin reports + event list
│   │   ├── public.routes.js        # Public clinic-info + health
│   │   ├── public-content.routes.js# Public treatments + articles
│   │   ├── enquiry.routes.js       # Public enquiry form (POST /enquiry)
│   │   └── seo.routes.js           # sitemap.xml, robots.txt, redirect
resolver
│   ├── controllers/            # Thin layer: validate → call service →
res.json
│   │   ├── authController.js, clinicInfoController.js, mediaController.js
│   │   ├── articleController.js, treatmentController.js
│   │   ├── testimonialController.js, galleryController.js,
videoController.js
│   │   ├── curatedPostController.js, announcementController.js,
homepageController.js, leadController.js
│   │   ├── enquiryController.js, practoController.js, profileController.js
│   │   ├── redirectController.js, seoController.js, analyticsController.js
│   ├── services/               # Business logic + DB writes
│   │   ├── authService.js          # Login, token issue/rotate/revoke
│   │   ├── clinicInfoService.js    # Clinic singleton read/update
(allowlisted)
│   │   ├── leadService.js          # Lead CRUD, stats, CSV export
│   │   ├── mediaService.js         # Cloudinary signed upload, confirm,
delete, list
│   │   └── seoService.js           # Sitemap, robots.txt, redirect lookup (+
unused JSON-LD builders)
│   ├── models/                 # Mongoose schemas (18 models, see Section 6)
│   │   └── index.js                # Re-exports all models
│   ├── middlewares/
│   │   ├── auth.middleware.js      # JWT check + optionalAuth (single admin,
no roles)
│   │   ├── validate.middleware.js  # express-validator chains + sanitizers
│   │   ├── rateLimit.middleware.js # 4 limiters (publicWrite/read, click,
auth)
│   │   └── errorHandler.middleware.js # Error classes, 404 + central error
handler, asyncHandler
│   ├── utils/
```

```
│   │   └── logger.js              # Winston logger
│   ├── integrations/
│   │   ├── googleAnalytics/       # EMPTY — no server GA code
│   │   └── practo/                # EMPTY — Practo is URL links only
│   └── jobs/                      # EMPTY — no cron jobs (notification is a
console.log stub)
├── infra/backup-scripts/backup.js  # Standalone mongodump backup script (not
part of the API)
├── tests/                        # jest + supertest integration tests (in-
memory Mongo)
├── docs/
│   ├── HANDOVER.md                # Older handover doc (may be stale — this
file is authoritative)
│   └── BACKEND_DOCUMENTATION.md   # This file
├── .env / .env.example           # Local env / template (never commit real
secrets)
└── package.json                  # Deps + jest config + scripts
```

Important files: - `src/app.js` — start here. Shows every route mount, CORS allowlist, cache headers, body limit (`1mb`), and startup. - `src/config/env.js` — every env var the code actually reads. - `src/middlewares/auth.middleware.js` — how the token is checked (`Authorization: Bearer <token>`). - `src/services/authService.js` — login/lockout/refresh logic (2FA fully removed). - `src/services/mediaService.js` — the 2-step Cloudinary upload flow the frontend must follow.

---

4. Architecture & Request Flow

A request travels like this:

```
Client
  → helmet / cors (allowlist) / compression / cookieParser / express.json
(1mb limit)
  → mongoSanitize + sanitizeBodyKeys (strips $ / . keys, trims strings)
  → Cache-Control setter (public/admin prefixes)
  → request logger (winston)
  → rate limiter (public + auth routes)
  → route file → validators (422 on failure) → authMiddleware (admin routes)
  → controller → service → Mongoose model → MongoDB
  → res.json(...)  (errors → central errorHandler → { error, code })
```

Text diagram:

```
+----------------+     +-------------------+     +------------------+
|    Frontend    |---->|  Express (app.js) |---->| Route + validator|
| (public site / |     |  prefix: /api/    |     | + authMiddleware |
|  admin panel)  |     |  public | admin   |     | (admin only)     |
+----------------+     +-------------------+     +--------+---------+
                                                          |
                                    +---------------------+--------------------
-+
```

```
                                    |
|
                           Controller (thin)
Service (logic)
                            - read req.body/query                  -
allowlist filtering, sanitizing
                            - call service                         -
Model.statics (queries)
                            - res.json()                           -
token/session handling (auth)
                                                                          |
                                                                       Mongoose
models
                                                                          |

MongoDB
```

Other notes: - **No service layer for some resources.** Testimonials, gallery, videos, curated posts, profile, redirects, and analytics do DB work directly in the controller. Only auth, clinic-info, leads, media, and SEO have services. - **No audit trail (removed by owner decision).** Admin writes do not log before/after snapshots anywhere. - **No background workers.** `src/jobs/` is empty; `node-cron` is installed but never imported. The "new enquiry notification" is a `setImmediate` + redacted `console.log` (lead id only, no PII) plus a persisted `AnalyticsEvent` (`enquiry_submit`). - **Every response is JSON**, except `GET /leads/export` (CSV download, streamed), `GET /sitemap.xml` (XML), `GET /robots.txt` (text), and `GET /redirect/:path` (HTTP 301/302 redirect).

5. Authentication & Authorization

Model: single-role admin. There is no signup and no patient/user account. One `users` collection holds admin logins; every admin has identical full permissions.

How login works (2FA removed — email + password only)

1. `POST /api/admin/auth/login` with `{ email, password }`.
2. Server checks: user exists → status is `active` → account not locked → bcrypt password matches.
3. On success: login attempts reset, a token pair is issued, the refresh-token hash is stored in DB. (No audit entry — audit logging removed.)
4. Response contains `{ user, accessToken, refreshToken }`, and the refresh token is also set as an `httpOnly` cookie (`refreshToken`, 7 days, `SameSite=Lax`, `Secure` in production).
5. Wrong password increments `loginAttempts`. After **5 failures the account locks for 15 minutes**; further attempts return `429`. A lock that already expired is cleared on the next attempt.
6. Unknown users, wrong passwords, and disabled accounts all return identical `401 { error: "Invalid credentials" }` (no account-state oracle).

Token type and lifetime

- Access token: JWT (`HS256`), default **15 minutes** (`JWT_ACCESS_EXPIRY`). Payload: `{ sub (user id), email, jti }`.

- Refresh token: JWT, default **7 days** (`JWT_REFRESH_EXPIRY`), **rotating**: each `POST /auth/refresh` revokes the old stored hash and stores a new one (DB `expiresAt` is hardcoded to 7 days from issue). Reusing an old refresh token returns `401`.
- Logout (`POST /auth/logout`) marks the refresh token revoked and clears the cookie. There is no access-token blacklist — a stolen access token stays valid until it expires (max 15 min by default).

Where to send the token

- Header name: `Authorization`. Format: `Authorization: Bearer <accessToken>`.
- `authMiddleware` (`src/middlewares/auth.middleware.js`) verifies the JWT, loads the user, rejects disabled users (`401`), and attaches `req.user`, `req.permissions`, `req.tokenPayload`.
- `optionalAuth` attaches the user only if a valid token is present (used on public clinic-info; harmless to always send the token).
- `authMiddleware` (`src/middlewares/auth.middleware.js`) verifies the JWT, loads the user, rejects disabled users (`401`), and attaches `req.user`. Single admin: a valid session is allowed everywhere, no roles or permissions.

Public vs protected (summary)

- Public (no token): `/api/public/*`, `POST /api/admin/auth/login`, `POST /api/admin/auth/refresh`, `POST /api/admin/auth/logout`, plus (by mount order) `GET /api/admin/booking-cta` and `POST /api/admin/leads/practo-click` (same handlers as the public ones).
- Protected (token required): everything else under `/api/admin/*`, including `GET /api/admin/auth/me` (FIXED: now behind `authMiddleware`).
- FIXED: testimonial/gallery/profile/curatedPost routers use split public/admin routers, so admin list filters work and public handlers no longer shadow them.

---

6. Database Models / Schema

18 Mongoose models (`AuditLog` and `ContentReview` deleted in the review; `Announcement` added after). Dates are stored as `Date` and serialized as ISO-8601 strings. `_id` is ObjectId hex string; `createdAt`/`updatedAt` come free with every model (`timestamps: true`).

Relationships in plain language: Treatments are the hub — testimonials, gallery items, and leads can point at a treatment; articles/videos/curated posts stand alone. Media (`MediaAsset`) holds every uploaded file, referenced by id from treatments, profiles, videos (captions), and articles (SEO image). `HomepageConfig`, `ClinicInfo`, `Profile`, and `BookingProviderConfig` are singletons (one document each). `Location` exists as a schema with `locationId` fields on all clinic-bound collections (Treatment, Article, Testimonial, GalleryItem, Lead, Video, CuratedPost, ClinicInfo, Profile), but **has no API endpoints**.

User (admins)

Field	Type	Required	Default / notes
`name`	String (max 100)	Yes	—

Field	Type	Required	Default / notes
`email`	String, unique, lowercase, indexed	Yes	login key
`passwordHash`	String, `select: false`	Yes	bcrypt(12); compared via `comparePassword()`
`status`	Enum `active`, `disabled`, indexed	No	default `active`
`lastLogin`	Date	No	set on successful login
`loginAttempts`	Number	No	default `0`; lock at 5
`lockUntil`	Date	No	now + 15 min when threshold hit
`isLocked` (virtual)	Boolean	—	true when `lockUntil` is in the future

RefreshToken

Field	Type	Required	Notes
`tokenHash`	String, unique, indexed	Yes	SHA-256 of the JWT (raw token never stored)
`userId`	ObjectId → User, indexed	Yes	—
`expiresAt`	Date (TTL index only)	Yes	auto-deleted by Mongo after expiry
`device`	String	No	default `''`. ⚠️ Service writes `userAgent`, **not** `device` — this field always stays empty
`ip`	String	No	default `''`
`revoked`	Boolean, indexed	No	default `false`

Methods: `isValid()` (not revoked + not expired), `revoke()`.

ClinicInfo (singleton, `singleton: "clinic_info"` unique)

Field	Type	Required	Notes
`practiceName`	String (max 200)	Yes	e.g. “Dr. Ekta Narula Dental Care”
`address`	Object `{ street, city, state,`	No	`country` defaults `"India"`, rest `""`

Field	Type	Required	Notes
	`pincode, country }`		
`phones`	String[]	No	each must match `/^[\d\s\-\+\(\)]{10,20}$/`
`timings`	Array of `{ day, open, close, isClosed }`	No	`day` enum Mon–Sun; `open`/`close` `"HH:mm"`
`mapCoordinates`	`{ latitude, longitude }`	No	default `0, 0`
`socialLinks`	Array of `{ platform, url }`	No	platform enum `instagram, youtube, facebook, linkedin, twitter`
`legalEntityName`	String	No	default `""`
`email`	String (lowercase)	No	default `""`
`locationId`	ObjectId → Location, indexed	No	default `null` (added in review; settable via PUT whitelist)

Statics: `getInstance()` (creates with defaults if missing), `updateSingleton(data)` (assign + save; no audit).

MediaAsset (every uploaded file)

Field	Type	Required	Notes
`filename` / `originalName`	String	Yes	—
`storageKey`	String, indexed	Yes	Cloudinary public id
`url`	String	Yes	original delivery URL
`optimizedUrl`	String	No	`f_auto,q_auto` delivery URL (images; derived server-side)
`variants`	Array of `{ name (thumb/medium/large), width (400/800/1600), url }`	No	responsive renditions (images only)
`url`	String	Yes	public delivery URL

Field	Type	Required	Notes
`mimeType`	Enum: `image/jpeg`, `image/png`, `image/webp`, `image/gif`, `video/mp4`, `video/webm`, `application/pdf`, `text/vtt`	Yes	size limits: images 5 MB, gif 3 MB, video 50 MB, pdf 10 MB, vtt 1 MB
`size`	Number (bytes, min 0)	Yes	validated by `validateFile()`
`uploadedBy`	ObjectId → User, indexed	Yes	—
`usedIn`	Array of `{ collection, documentId, field }`	No	usage tracking — ⚠️ nothing in the code ever writes it, so it is always empty
`altText`	String	No	default `""`

Statics: `validateFile(mimeType, size)`, `isInUse` (true when `usedIn` is non-empty; entries written directly).

ContentReview — REMOVED (review)

The review-queue sidecar (`ContentReview` model + `contentApprovalService`) was deleted in the review: content is `draft`/`published` only. There is no `pending/approved/rejected` state anywhere.

Treatment (treatment pages, draft/published)

Field	Type	Required	Notes
`slug`	String, unique, lowercase, `/^[a-z0-9-]+$/`, max 100, indexed	Yes	public URL key
`title`	String (max 200)	Yes	text-indexed
`status`	Enum `draft`, `published`, indexed	No	default `draft`; set directly by admin, no workflow
`locationId`	ObjectId → Location	No	default `null` (no location API)
`sections`	Object `{ what, who, procedure, pain, recovery, risks }` (all optional strings)	No	page body sections; HTML sanitized on write
`faqs`	Array of `{ question (req,`	No	answers sanitized on write

Field	Type	Required	Notes
	max 500), answer (req), order (default 0) }		
heroImage	ObjectId → MediaAsset	No	—
gallery	ObjectId[] → MediaAsset	No	—
relatedTreatments	ObjectId[] → Treatment	No	populated as title, slug
seo	{ metaTitle (max 60), metaDescription (max 160), ogImage (→ MediaAsset) }	No	—
createdBy	ObjectId → User	Yes	required; excluded from public payloads
publishedAt	Date	No	stamped automatically when status becomes published

Statics: getPublished (paginated, populated, optional $text search; .lean(), excludes createdBy), getBySlug (published only, excludes createdBy), getAllForAdmin (status + regex search), stampPublishedAt(doc).

Article (knowledge-centre posts, draft/published)

Field	Type	Required	Notes
title	String (max 200)	Yes	text-indexed
slug	String, unique, lowercase, /^[a-z0-9-]+$/, max 100	Yes	public URL key
category	Enum preventive-care, restorative, cosmetic, orthodontics, oral-surgery, pediatric, emergency, technology, patient-education, indexed	Yes	—
tags	String[] (lowercased)	No	text-indexed

Field	Type	Required	Notes
`body`	String	Yes	full HTML/text — sanitized on write; ⚠️ list endpoint omits it (`.select('-body')`), detail includes it
`author`	String (max 100)	Yes	free-text name
`reviewer`	ObjectId → User	No	populated as `name` (public)
`credentialsDisplay`	String (max 500)	No	e.g. “MDS, Orthodontics”
`sources`	Array of `{ title, url, authors, publishedDate }`	No	—
`status`	Enum `draft`, `published`, indexed	No	default `draft`; set directly by admin, no workflow
`locationId`	ObjectId → Location	No	default `null`
`publishedAt`	Date	No	stamped automatically when status becomes `published`
`seo`	Same shape as Treatment	No	—

Statics: `getPublished` (filters category/tags/search; body excluded; `.lean()`), `getBySlug` (`.lean()`), `getAllForAdmin`, `stampPublishedAt(doc)`.

Lead (CRM rows from forms and clicks)

Field	Type	Required	Notes
`name`	String (max 100)	Yes	—
`phone`	String, indexed	Yes	format validated at the edges, not in schema
`treatmentInterest`	ObjectId → Treatment	No	default `null`
`source`	Enum `enquiry_form`, `practo_click`, `phone_click`, indexed	Yes	`phone_click` is never written by any code today
`locationId`	ObjectId → Location	No	default `null`

Field	Type	Required	Notes
`date`	Date, indexed	No	default `Date.now`
`leadStatus`	Enum `new`, `contacted`, `scheduled`, `completed`, `lost`, indexed	No	default `new`
`appointmentStatus`	Enum `none`, `booked`, `completed`, `cancelled`, `no_show`	No	default `none`
`notes`	String (max 1000)	No	—

Statics: `getLeads` (filters status/source/date-range/search + pagination; sanitized-empty search is ignored), `updateLead` (only `leadStatus, appointmentStatus, notes, treatmentInterest` are writable), `exportCursor` (lean streaming cursor for CSV).

BookingProviderConfig (singleton, `singleton: "booking_provider"` unique)

Field	Type	Required	Notes
`providerName`	Enum `practo`	No	default `practo`
`profileUrl` / `bookingUrl`	String	No	default `""` — the URLs the frontend "Book" button uses
`isActive`	Boolean	No	default `true`; public CTA 404s when no active config

Statics: `getActive()`, `getOrCreate()`.

Profile (doctor singleton — no approval workflow)

Arrays of `qualifications [{ degree, institution, year }]`, `memberships [{ organization, role, since }]`, `awards [{ title, issuer, year }]`, `mediaAppearances [{ outlet, title, date, url }]`, `publications [{ title, journal, year, url }]` (years 1950–2030), plus `photo` (→ MediaAsset), `video` (→ MediaAsset), `bio` (String, sanitized on write), `locationId` (→ Location, default `null`). Statics: `getSingleton()` (creates empty doc if missing, populates photo/video), `updateSingleton(data, userId)` (whitelist of the 9 fields).

Testimonial (patient stories, draft/published)

Field	Type	Required	Notes
`patientDisplayName`	String (max 100)	Yes	—

Field	Type	Required	Notes
`treatmentTag`	ObjectId → Treatment	No	populated as `title, slug`
`type`	Enum `text, video, audio`	No	default `text`
`content`	String	Yes	sanitized on write
`status`	Enum `draft, published`, indexed	No	default `draft`; set directly by admin, no workflow, no consent gate
`locationId`	ObjectId → Location	No	default `null`
`publishedAt`	Date	No	stamped automatically when status becomes `published`

Statics: `getPublished` (filter `treatmentTag`; `.lean()`), `getAllForAdmin` (filters status/treatmentTag), `stampPublishedAt(doc)`.

GalleryItem (before/after pairs, draft/published)

`beforeImg` (→ MediaAsset, required), `afterImg` (→ MediaAsset, required), `treatmentTag` (→ Treatment), `content` (optional caption, sanitized on write), `status` enum `draft, published` (default `draft`, set directly), `locationId`, `publishedAt` (auto-stamped). No consent fields, no review fields. Statics mirror Testimonial (`getPublished` with `.lean()`, `getAllForAdmin`), populating both images.

Video (YouTube embeds — plain CRUD, no approval workflow)

Field	Type	Required	Notes
`url`	String	Yes	must be a URL (validated)
`title`	String (max 200)	Yes	—
`category`	String (max 100), indexed	Yes	free text, no enum
`thumbnail`	String	No	must be URL if present
`order`	Number, indexed	No	default `0`; public sort is `order` then newest
`status`	Enum `published, hidden`, indexed	No	default `published` (different vocab from other content!)
`locationId`	ObjectId → Location	No	default `null`

Field	Type	Required	Notes
`transcript`	String	No	accessibility text
`captionsUrl`	ObjectId → MediaAsset	No	VTT file

Statics: `getPublished` (filter category), `getAllForAdmin` (filters status/category).

CuratedPost (manual Instagram grid — plain CRUD)

Field	Type	Required	Notes
`image`	ObjectId → MediaAsset	Yes	—
`caption`	String	No	—
`permalink`	String	Yes	must be URL (link to Instagram)
`order`	Number, indexed	No	default `0`
`locationId`	ObjectId → Location	No	default `null`

No status field — every post is immediately public. Statics: `getPublished`, `getAllForAdmin` (identical queries; both populate `image`).

Announcement (clinic notices, draft/published)

Field	Type	Required	Notes
`title`	String (max 200)	Yes	—
`body`	String	Yes	sanitized on write
`linkUrl`	String	No	must be URL if present (e.g. link to a treatment page)
`status`	Enum `draft`, `published`, indexed	No	default `draft`; set directly by admin
`order`	Number, indexed	No	default `0`; public list sorts by `order`, then newest
`locationId`	ObjectId → Location	No	default `null`
`publishedAt`	Date	No	stamped automatically when status becomes `published`

Statics: `getPublished` (ordered, `.lean()`), `getAllForAdmin` (status filter), `stampPublishedAt(doc)`. Public: `GET /api/public/announcements` (ordered published list). Admin: full CRUD at `/api/admin/announcements`.

HomepageConfig (singleton, no `singleton` key — first document wins)

`featuredTreatments[]` (→ Treatment), `featuredArticles[]` (→ Article), `featuredVideos[]` (→ Video), `featuredTestimonials[]` (→ Testimonial). Statics: `getSingleton()` (creates `{}` if missing), `updateSingleton(data)` (whitelist of the 4 arrays). Public homepage shows featured first, then latest, capped at 6/4/4/3.

RedirectMap

Field	Type	Required	Notes
`fromPath`	String (max 500), unique	Yes	controller prepends `/` if missing on create
`toPath`	String (max 500)	Yes	—
`statusCode`	Enum `301, 302`	No	default `301`

Statics: `findByFromPath`, `getAllForAdmin` (search across both paths + pagination).

AnalyticsEvent (server-side event log)

`eventType` enum `page_view, enquiry_submit, practo_click, appointment_book, video_play, testimonial_view, gallery_view, article_read, treatment_view` (indexed, required); `refId` (ObjectId) + `refModel` enum `Treatment, Article, Video, Testimonial, GalleryItem, Lead, Appointment` (⚠️ no `Appointment` model exists); `metadata` (Mixed, default `{}`); `sessionId`, `ip`, `userAgent` (optional strings). Statics: `logEvent` / `getMonthlyReport` (by-event-type counts, daily counts, top `refId`s, enquiry sources by `metadata.source`). Writers (review): `enquiry_submit` on every enquiry, `practo_click` on every click log — the monthly report is live data now.

Location (no API endpoints — schema only)

`name` (req, max 100), `code` (req, unique, uppercase, `/^[A-Z0-9-]+$/`, max 20), `address` (street/city/state/pincode/country default India), `phones[]` (same phone regex), `timings[]` (day/open/close/isClosed), `mapCoordinates` (lat −90..90, lng −180..180), `isActive` (default true, indexed), `isDefault` (default false). Statics: `getDefault`, `getActive`, `getByCode` — none reachable via HTTP.

AuditLog — REMOVED (review)

Audit logging was deleted by owner decision (single admin). There is no `AuditLog` model, no `GET /api/admin/audit-log`, and no before/after snapshots anywhere.

7. Complete API Reference

Conventions: `{{base}}` = `http://localhost:4000`. `{{token}}` = admin access token. All bodies are JSON unless stated. `Auth` = `Authorization: Bearer {{token}}` required, or `No` for public.

7.1 Root, health, clinic info (public)

`GET /` — **API info.** Auth: No. Success `200`: `{ "name": "Dr. Ekta Narula Dental Care API", "version": "1.0.0", "status": "running" }`. Curl: `curl {{base}}/`. Frontend note: use for "backend reachable?" checks.

`GET /api/public/health` — **Health check.** Auth: No. Rate-limited (100/min). Success `200`: `{ "status": "ok", "timestamp": "2026-09-21T00:00:00.000Z" }`. Curl: `curl {{base}}/api/public/health`. Frontend note: uptime monitor / load-balancer probe.

`GET /api/public/clinic-info` — **Clinic contact, timings, socials.** Auth: No (optional token accepted, changes nothing). Cached 1 hour. Success `200`: the ClinicInfo document, e.g.

```
{
  "_id": "66f...",
  "practiceName": "Dr. Ekta Narula Dental Care",
  "address": { "street": "", "city": "", "state": "", "pincode": "",
        "country": "India" },
  "phones": ["+91-..."], "timings": [{ "day": "monday", "open": "10:00",
        "close": "20:00", "isClosed": false }],
  "mapCoordinates": { "latitude": 0, "longitude": 0 },
  "socialLinks": [{ "platform": "instagram", "url": "https://..." }],
  "legalEntityName": "", "email": "",
  "createdAt": "...", "updatedAt": "..."
}
```

Errors: `500` on DB failure. Curl: `curl {{base}}/api/public/clinic-info`. Frontend note: powers header/footer/contact page (phones, address, hours, map pin, social icons). No params; single object, not a list.

7.2 Treatments & articles (public)

`GET /api/public/treatments` — **Published treatment list.** Auth: No. Cached 5 min. Query: `page` (≥1, default 1) | `limit` (1–100, default 20) | `search` (full-text on title + sections). Success `200`: `{ "items": [Treatment…], "pagination": { "page": 1, "limit": 20, "total": 5, "pages": 1 } }` (items populate `heroImage`, `gallery`, `relatedTreatments{title,slug}`). Curl: `curl "{{base}}/api/public/treatments?page=1&limit=20"`. Frontend note: treatment listing page + cards. `items[].slug` links to the detail page.

`GET /api/public/treatments/:slug` — **Treatment detail.** Auth: No. Cached 5 min. Path: `slug` (e.g. `root-canal`). Success `200`: full Treatment incl. `sections`, `faqs`, populated images, `seo`. Error: `404 { error, code: NOT_FOUND }` when slug unknown/unpublished. Curl: `curl {{base}}/api/public/treatments/root-canal`. Frontend note: treatment detail page; render `sections.*`, FAQ accordion, `seo.metaTitle/metaDescription` into `<head>`.

`GET /api/public/articles` — **Published article list.** Auth: No. Cached 5 min. Query: `page` | `limit` | `category` (must be one of the 9 enum values) | `tags` (repeatable, e.g. `?tags=x&tags=y`) | `search` (full-text title/body/tags). ⚠️ Response items **exclude `body`** (list

is teaser-only by design). Curl: `curl "{{base}}/api/public/articles?category=preventive-care&page=1&limit=9"`. Frontend note: blog listing; link each card to `/articles/:slug` for the full body.

`GET /api/public/articles/:slug` — Article detail (full body, sources, SEO). Auth: No. Cached 5 min. Error: `404 { error, code: NOT_FOUND }` when slug unknown/unpublished. Curl: `curl {{base}}/api/public/articles/my-article`. Frontend note: blog detail page.

7.3 Profile, homepage, testimonials, gallery, videos, Instagram (public)

`GET /api/public/profile/` — Doctor bio. Auth: No. Cached 1 hour. Success `200`: Profile singleton with `photo`/`video` populated (MediaAsset objects or null). Curl: `curl {{base}}/api/public/profile/`. Frontend note: About page (qualifications, awards, bio, photo).

`GET /api/public/home/` — Homepage aggregate. Auth: No. Cached 1 min. Success `200`:

```
{ "treatments": [/* max 6 */], "articles": [/* max 4, no body */],
  "videos": [/* max 4 */], "testimonials": [/* max 3 */] }
```

Featured (admin-pinned) items come first, then latest published. No params. Curl: `curl {{base}}/api/public/home/`. Frontend note: powers the entire homepage in **one call** — do not fetch the four lists separately.

`GET /api/public/testimonials/` — Published patient stories. Auth: No. Query: `page` (validated ≥1) | `limit` (validated 1–100) | `treatmentTag` (Treatment ObjectId, optional). Success `200`: `{ items, pagination }`, items populate `treatmentTag{title,slug}`. Curl: `curl "{{base}}/api/public/testimonials/?treatmentTag=66f..."`. Frontend note: testimonial carousel/section; filter by treatment on treatment pages.

`GET /api/public/gallery/` — Published before/after items. Auth: No. Query: same as testimonials (`page`, `limit`, `treatmentTag`). Success `200`: `{ items, pagination }`, items populate `beforeImg`, `afterImg`, `treatmentTag{title,slug}`. Curl: `curl {{base}}/api/public/gallery/`. Frontend note: smile-gallery with before/after slider (`beforeImg.url`, `afterImg.url`).

`GET /api/public/videos/` — Published videos. Auth: No. Query: `page` | `limit` | `category` (free-text category, exact match). Success `200`: `{ items, pagination }`, items populate `captionsUrl`. Item shape: `{ _id, url, title, category, thumbnail, order, status, transcript, captionsUrl, createdAt, updatedAt }`. Curl: `curl "{{base}}/api/public/videos/?category=education"`. Frontend note: video library; embed `url` (YouTube), show `transcript` for accessibility.

`GET /api/public/instagram-posts/` — Curated Instagram grid. Auth: No. Query: `page` (validated) | `limit` (validated). No status filter exists — everything stored is public. Success `200`: `{ items, pagination }`, items populate `image` (MediaAsset). Curl: `curl {{base}}/api/public/instagram-posts/`. Frontend note: Instagram strip; each item links out to `permalink`.

7.4 Enquiry, booking CTA, click log (public writes)

`POST /api/public/enquiry` — Patient enquiry form. Auth: No. Strictly rate-limited (20/min/IP+path). Body:

Field	Type	Required	Rules
`name`	string	Yes	1–100 chars
`phone`	string	Yes	10–20 chars, `/^[\d\s\-\+\(\)]{10,20}$/`
`treatmentInterest`	ObjectId string	No	must be an existing Treatment id if sent
`honeypot`	string	No	must be empty — bots filling it get a fake success
`captchaToken`	string	No	verified server-side against reCAPTCHA when `RECAPTCHA_SECRET` is set (5 s timeout, fail-closed); without a secret only the literal `"invalid-token"` is rejected (422)
medical fields (`symptoms`, `diagnosis`, `medicalHistory`, `allergies`, `medications`, `conditions`)	—	**Forbidden**	sending any returns 422 (patients must not submit health data)

Success `201`: `{ "message": "Enquiry submitted successfully", "leadId": "66f..." }`. Errors: `422` validation (also for bad phone, unknown treatment, medical fields); `429` rate limit; honeypot-filled returns fake `200 { message }` with no lead created. Curl:

```
curl -X POST {{base}}/api/public/enquiry -H "Content-Type: application/json"
          \
  -d '{"name":"Aarav","phone":"+91-9876543210","treatmentInterest":"
        <treatmentId>","honeypot":""}'
```

Frontend note: the contact/enquiry form. Keep a hidden `honeypot` input; never add symptom/diagnosis fields. Save nothing — show success message.

`GET /api/public/booking-cta` — Practo booking links. Auth: No. Success `200`: `{ "provider": "practo", "profileUrl": "https://…", "bookingUrl": "https://…" }`. Error: `404 { error: "No active booking provider configured" }` — frontend must hide/fallback the Book button then. Curl: `curl {{base}}/api/public/booking-cta`. Frontend note: “Book Appointment” button href = `bookingUrl`. (Same handler also serves `GET /api/admin/booking-cta` — use the public one.)

`POST /api/public/leads/practo-click` — **Fire-and-forget click log.** Auth: No. Body ignored (send `{}`). Always returns `200 { message: "Click logged" }`, even on DB error. Creates a Lead `{ name: "Practo Click", phone: "unknown", source: "practo_click" }`. Curl: `curl -X POST {{base}}/api/public/leads/practo-click -H "Content-Type: application/json" -d '{}'`. Frontend note: call (don't await) when the user clicks the Practo/book button. (Also reachable as `POST /api/admin/leads/practo-click`.)

7.5 SEO (public)

`GET /api/public/sitemap.xml` — **Generated sitemap.** Auth: No. Cached 1 hour. Content-Type `application/xml`. Lists `/`, `/treatments`, `/articles`, `/videos`, `/gallery`, `/testimonials`, `/about`, `/contact`, `/booking` plus every published treatment/article URL and video URLs by id (`/videos/<_id>` — Video has no slug). XML entities escaped properly. Base URL = `FRONTEND_URL` (fallback `https://drekta.com`). Curl: `curl {{base}}/api/public/sitemap.xml`.

`GET /api/public/robots.txt` — **Generated robots.** Auth: No. Disallows `/api/` and `/admin/`, points at the sitemap. Curl: `curl {{base}}/api/public/robots.txt`.

`GET /api/public/redirect/:path(*)` — **Legacy-URL redirect.** Auth: No. Looks up `RedirectMap` by `fromPath` (`/<path>`); success issues HTTP `301`/`302` to `toPath`. Unknown path → `404`. Curl: `curl -v {{base}}/api/public/redirect/old-page`. Frontend note: rarely called by the SPA directly; kept for server-side/link-compat use.

7.6 Admin auth (no token needed)

`POST /api/admin/auth/login` — **Admin login.** Auth: No. Rate-limited (10 per 15 min). Body: `{ "email": "admin@…", "password": "…" }` (email validated/normalized; no other fields — 2FA removed). Success `200`:

```
{ "user": { "id": "66f…", "name": "…", "email": "…" },
  "accessToken": "<jwt-15m>", "refreshToken": "<jwt-7d>" }
```

(Refresh token also set as httpOnly cookie.) Errors: `401 { error: "Invalid credentials" }` (unknown user, bad password, or disabled account — all identical); `429 { error: "Account temporarily locked. Try again later." }` (5 failed attempts → 15-min lock); `422` validation. Curl:

```
curl -X POST {{base}}/api/admin/auth/login -H "Content-Type:
         application/json" \
  -d '{"email":"admin@example.com","password":"<password>"}'
```

Frontend note: admin login screen. Store `accessToken` in memory and `refreshToken` (body value) securely; call `/auth/me` or any admin endpoint to verify.

`POST /api/admin/auth/refresh` — **Rotate tokens.** Auth: No (the refresh token *is* the credential). Rate-limited. Body `{ "refreshToken": "…" }`, or the httpOnly cookie is read automatically. Success `200`: `{ accessToken, refreshToken }` (new pair; old refresh token revoked). Errors: `401` missing/invalid/revoked/expired token or disabled user. Curl: `curl -X POST {{base}}/api/admin/auth/refresh -H "Content-Type: application/json" -d '{"refreshToken":"<rt>"}'`. Frontend note: axios interceptor — on any `401` from an admin call, refresh once and retry; if refresh fails, redirect to login.

`POST /api/admin/auth/logout` — Revoke refresh token. Auth: No. Body `{ refreshToken }` optional (or cookie); always `200 { message: "Logged out successfully" }`, clears cookie. Frontend note: logout button — also discard stored tokens client-side (access tokens stay technically valid until expiry).

`GET /api/admin/auth/me` — Current admin. Auth: Yes (FIXED: now behind `authMiddleware`). Success `200 { user: { id, name, email, lastLogin } }`. Frontend note: safe to use for session verification.

7.7 Admin clinic-info and media

`GET /api/admin/clinic-info` (Auth: Yes) — same document as public, no cache. **`PUT /api/admin/clinic-info`** (Auth: Yes) — update whitelisted fields (`practiceName, address, phones, timings, mapCoordinates, socialLinks, legalEntityName, email, locationId`); unknown fields ignored; returns updated doc. No audit entry (audit removed). Validation: phones regex, `HH:mm` timings, day/platform enums, lat/lng ranges, `socialLinks[].url` must be URL, `email` must be email. Errors: `422`. Curl: `curl -X PUT {{base}}/api/admin/clinic-info -H "Authorization: Bearer {{token}}" -H "Content-Type: application/json" -d '{"phones":["+91-9876543210"]}'`. Frontend note: clinic settings screen.

`POST /api/admin/media/upload` (Auth: Yes) — Step 1 of upload: validate + get Cloudinary signature. Body: `{ filename, mimeType, size }` (mimeType must be in the allowed list, size within its cap — see MediaAsset). Success `200`: `{ uploadUrl, publicUrl, optimizedUrl, variants, publicId, signature, timestamp, apiKey, cloudName, resourceType, paramsToSign, expiresIn: 3600 }`. Image pipeline: `paramsToSign.eager` carries 400/800/1600px `f_auto,q_auto` transforms so renditions generate at upload time; `optimizedUrl` + `variants[{name,width,url}]` are deterministic delivery URLs. Non-images get no eager/variants. Error: `422` for disallowed type/oversize. Frontend note: call first, then POST the file from the browser straight to `uploadUrl` (see integration guide). Render images via `optimizedUrl`/`variants` (thumb → cards, medium → content, large → fullscreen), never the raw `url`, for the §22 performance budget.

`POST /api/admin/media/confirm` (Auth: Yes) — Step 2: create the MediaAsset row. Requires **`key`** (checked by controller) **and `publicId`** (stored as `storageKey`) — send both with the same value — plus a valid http(s) `url`; type/size re-validated. `optimizedUrl` + `variants` are derived server-side from the public ID and stored on the asset. Success `201`: the MediaAsset. Error: `400` missing fields; `422` bad type/size/reference. Curl:

```
curl -X POST {{base}}/api/admin/media/confirm -H "Authorization: Bearer
        {{token}}" -H "Content-Type: application/json" \
  -d '{"filename":"smile.jpg","mimeType":"image/jpeg","size":12345,"key":"
        <publicId>","publicId":"<publicId>","url":"<publicUrl>"}'
```

`GET /api/admin/media` (Auth: Yes) — list uploads. Query `page` (validated), `limit` (validated), `mimeType` (exact match). ⚠️ Always filters to `uploadedBy = you`, so each admin only sees their own uploads. Returns `{ items, pagination }` with `uploadedBy{name,email}` populated.

`GET /api/admin/media/:id` (Auth: Yes) — one asset (`:id` must be ObjectId, else 422; unknown → 404).

`DELETE /api/admin/media/:id` (Auth: Yes) — deletes from Cloudinary (best-effort) + DB. The "in use" guard (`409 { error, inUse: true }`) exists but **never triggers** because nothing ever populates `usedIn` — treat deletes as unguarded.

7.8 Admin treatments + articles (draft/published, no workflow)

Treatments (admin). Base `/api/admin/treatments` (Auth: Yes for all). `status` is `draft` or `published`, set directly on create/update; `publishedAt` auto-stamps on publish. Unknown `status` values → `422`.

Method + path	Purpose	Body / notes
`GET /`	List all statuses. Query `page, limit` (capped 100), `status, search` (regex on title/slug)	`{ items, pagination }`, populates `heroImage, createdBy`
`GET /:id`	One treatment by ObjectId	`404` when unknown
`POST /`	Create (draft or published). `createdBy` set automatically	`title` req; `slug` optional `/^[a-z0-9-]+$/`; sections/faqs/heroImage/gallery/relatedTreatments/seo validated; only whitelisted fields stored. `201`
`PUT /:id`	Edit content fields incl. direct `status`	allowlisted; rich-text sanitized; `404` when unknown
`DELETE /:id`	Delete	`{ message }`; `404` when unknown

Frontend note: CMS edit page with a Draft/Published toggle; no Submit/Approve/Reject buttons.

Articles (admin). Base `/api/admin/articles` (Auth: Yes). Same shape as treatments: `GET /` (`page, limit` capped, `status, category, search`), `GET /:id` (`404` when unknown), `POST /` (`title, category, body, author` required; `201`), `PUT /:id` (allowlisted incl. direct `status`; reviewer/credentialsDisplay/sources editable), `DELETE /:id` (`404` when unknown).

7.9 Admin testimonials + gallery (draft/published, no consent gate)

Testimonials. Base `/api/admin/testimonials` (Auth: Yes). Split routers — admin list filtering works.

Method + path	Purpose	Notes
`GET /`	List all statuses (`status` filter works)	Same query as public (`page, limit` capped, `status, treatmentTag`)

Method + path	Purpose	Notes
`GET /:id`	One testimonial, populated `treatmentTag`	404 when unknown
`POST /`	Create (draft or published)	`patientDisplayName` + `content` required; content sanitized; `201`
`PUT /:id`	Edit incl. direct `status` (all-optional validator)	content sanitized; `publishedAt` auto-stamps; `404` when unknown
`DELETE /:id`	Delete	`404` when unknown

Gallery. Base `/api/admin/gallery` (Auth: Yes). Same shape, except create requires `beforeImg` + `afterImg` (MediaAsset ObjectIds) and update whitelist is `beforeImg, afterImg, treatmentTag, content, status, locationId` (all-optional validator). Frontend note: no consent collection needed — removed by owner decision.

7.10 Admin videos, Instagram posts, homepage config (plain CRUD)

Videos (Auth: Yes; no workflow — `status: published|hidden` set directly). `GET /api/admin/videos/` (`page, limit, status, category`), `GET /:id`, `POST /` (`url` must be URL, `title`, `category` required; transcript sanitized; `201` populated), `PUT /:id` (whitelist incl. `status`; note the update validator is still create-shaped — sending a partial body can 422, so send the full object on video edits), `DELETE /:id`. (Public counterpart: `GET /api/public/videos/`.)

Instagram posts (Auth: Yes; no status — everything is live). Split routers — admin list works. `GET /api/admin/instagram-posts/`, `GET /:id`, `POST /` (`image` MediaAsset + `permalink` URL required; `201`), `PUT /:id` (all-optional validator), `DELETE /:id`.

Announcements (Auth: Yes; `status: draft|published` set directly). `GET /api/admin/announcements/` (`page, limit` capped, `status`), `GET /:id` (`404` when unknown), `POST /` (`title`, `body` required; `linkUrl` must be URL if sent; `201`), `PUT /:id` (all-optional validator; `publishedAt` auto-stamps), `DELETE /:id`. Body sanitized. (Public counterpart: `GET /api/public/announcements/` — ordered published list.)

Homepage config (Auth: Yes for writes; reads oddly mounted). `GET /api/admin/home/` → serves the **public homepage aggregate** (public handler also mounted here; harmless duplicate, no auth needed). `GET /api/public/home/` → same aggregate. `GET /api/admin/home/config` → the raw config (requires token). `PUT /api/admin/home/config` → replace any of the four featured-id arrays (requires token; prefer the `/api/admin` one). Body: `{ featuredTreatments: [ids], featuredArticles: [ids], featuredVideos: [ids], featuredTestimonials: [ids] }`. Frontend note: "feature on homepage" pickers in each CMS section write id arrays here.

7.11 Admin leads

Base `/api/admin/leads` (Auth: Yes). ⚠️ Order matters: `/stats` and `/export` are registered before `/:id`, so the literal strings work.

Method + path	Purpose
`GET /`	List. Query `page, limit, status` (=`leadStatus`), `source`, `startDate`, `endDate`, `search` (name/phone regex). Returns `{ items, pagination }` with `treatmentInterest{title,slug}`
`GET /stats`	Funnel numbers: `{ byStatus: {…}, bySource: {…}, total, last7Days }`, optional `startDate, endDate, source`
`GET /export`	Streamed CSV download (`Content-Type: text/csv`, `Content-Disposition: attachment; filename="leads-YYYY-MM-DD.csv"`; Mongoose cursor, never buffered; cells starting with `= + - @` are formula-escaped). Query `startDate, endDate, source, status`. Columns: Name,Phone,Treatment Interest,Source,Date,Lead Status,Appointment Status,Notes,Created At
`GET /:id`	One lead (populated). `404` when unknown
`PUT /:id`	Update only `leadStatus, appointmentStatus, notes, treatmentInterest` (others ignored). Returns updated lead
`DELETE /:id`	Delete

Curl (stats): `curl "{{base}}/api/admin/leads/stats?source=enquiry_form" -H "Authorization: Bearer {{token}}"`. Frontend note: leads table + filters, status dropdown (`new→contacted→scheduled→completed/lost`), CSV export button (fetch as blob), dashboard cards from `/stats`.

7.12 Admin booking provider, redirects, analytics

`GET /api/admin/booking-provider` (Auth: Yes) — current Practo config or 404. **`PUT /api/admin/booking-provider`** (Auth: Yes) — upsert `{ providerName?, profileUrl?, bookingUrl?, isActive? }` (providerName forced to `"practo"` when omitted). **`GET /api/admin/practo-clicks`** (Auth: Yes) — `{ totalClicks, dailyStats: [{ _id: "YYYY-MM-DD", count }] }` (last 30 days), optional `startDate, endDate`. Frontend note: settings screen for the Book URLs + a tiny click counter. After saving, the public site reads the new URLs immediately (no cache on this path).

Redirects (Auth: Yes). `GET /api/admin/redirects/` (`page, limit` capped, `search`), `GET /:id` (404 unknown), `POST /` (`fromPath` req — `/` prepended if missing; duplicate → `409 DUPLICATE_REDIRECT`; `toPath` req; `statusCode` 301/302 default 301; `201`), `PUT /:id` (only `toPath`/`statusCode` editable — `fromPath` immutable), `DELETE /:id`. Frontend note: SEO redirect table. The public resolver only exact-matches `fromPath`.

`GET /api/admin/analytics/reports/monthly` (Auth: Yes) — `{ byEventType: {…}, dailyStats: [{date,count}], topPages: [{_id: refId, count}] (top 10), leadSources: {…} }`, optional `startDate, endDate`. Live data: enquiries log

`enquiry_submit` and booking clicks log `practo_click` automatically. **`GET /api/admin/analytics/events`** (Auth: Yes) — raw event list (`page`/`limit` capped, `eventType`, `startDate`, `endDate`) → `{ items, pagination }`.

That is every route: **~81 registrations** — ~62 under `/api/admin`, ~18 under `/api/public`, plus `GET /`. (Counts are lower than the 130 once claimed because the review deleted the audit, review-queue, and 18 workflow endpoints; `practo` and `homepage` routers are still mounted under **both** prefixes.)

8. Function & Module Reference

Format per row: **name** — what it does | params → returns | side effects | called from.

`src/app.js` / `src/config/*` / `src/utils/*` / `infra`

Function	File	What
`startServer`	`src/app.js`	Connects DB, listens on `env.port`, installs SIGTERM/SIGINT graceful shutdown
`shutdown` (inner)	`src/app.js`	Closes server + DB; force-exits after 10 s
anonymous middlewares	`src/app.js`	Cache-Control setter (public/admin), winston r logger
`connectDB`	`src/config/db.js`	Mongoose connect, pool min 10/max 20, retry timeouts; no-op if already connected
`disconnectDB`	`src/config/db.js`	Graceful disconnect
`getConnection`	`src/config/db.js`	Returns `mongoose.connection`
`validateEnv`	`src/config/env.js`	Throws if `MONGODB_URI`/`JWT_ACCESS_SECRET`/`JWT_REFRESH` missing (skipped in test)
`logger`	`src/utils/logger.js`	Winston console/JSON logger
`ensureDir`, `getTimestamp`, `runBackup`, `pruneBackups`, `main`	`infra/backup-scripts/backup.js`	`mongodump --gzip` to `infra/backups/daily`, k daily + 4 weekly (Sundays) + 12 monthly (1st) standalone script, not imported by the API

Middlewares

Function	File	What
`extractToken`	`auth.middleware.js`	Reads `Authorizati` `Bearer …` header
`authMiddleware`	`auth.middleware.js`	Verifies access JWT active user, attaches

Function	File	What
		`req.user`; 401 missing/bad/disable on unexpected erro
`optionalAuth`	`auth.middleware.js`	Same but never reje
`AppError` + 4 subclasses	`errorHandler.middleware.js`	`AppError(message, status, code, det` `ValidationError(4` `AuthenticationErr` `NotFoundError(404` `ConflictError(409`
`handleMongooseValidationError` / `handleDuplicateKeyError` / `handleCastError`	`errorHandler.middleware.js`	Normalize Mongoose/Mongo e `ValidationError`
`errorHandler`	`errorHandler.middleware.js`	Central 4-arg handl (console + winston) known errors, hides messages in prod (`I server error`), leal only in test
`notFoundHandler`	`errorHandler.middleware.js`	`404 { error: "Rou found", code: "NOT_FOUND" }`
`asyncHandler` (shared)	`errorHandler.middleware.js`	Wraps async fns → `next(err)`
`createStore` / `createRateLimiter` / `resetRateLimiterStore`	`rateLimit.middleware.js`	MemoryStore regis limiter factory (IP+ key, JSON 429, skij unless `x-enable-ra limit: true`); test r
limiters: `publicWriteLimiter` (20/min), `publicReadLimiter` (100/min), `authLimiter` (10/15 min)	`rateLimit.middleware.js`	Preconfigured limit
`handleValidationErrors`	`validate.middleware.js`	Returns `422 { erro "Validation faile details: [{field,message,v }` on express-valida failures
`sanitizeRichText`	`validate.middleware.js`	Allowlist HTML sa (p/br/strong/em/u/li h4/a/img) — called treatment sections/I article body, testimonial/gallery profile bio, video tr

Function	File	What
`sanitizeKeys` / `sanitizeBodyKeys`	`validate.middleware.js`	Recursively drops $ prefixed keys (NoS injection guard) + t strings; applied to body+query+param
`validators.*`	`validate.middleware.js`	Chains: `login`, `refreshToken`, `clir` `mediaUpload`, `pagir` (limit ≤ 100), `treat` `article`, `objectId(` factory, `publicEnqu` `profile`, `testimoni` `testimonialUpdate` `galleryItem`, `galleryItemUpdate` `curatedPost`, `curatedPostUpdate`

Models (statics, methods, hooks)

Function	File	What
`comparePassword`	`models/User.js`	bcrypt compare
pre-save hook	`models/User.js`	bcrypt(12)-hashes `passwordHash` when modified
`incrementLoginAttempts`	`models/User.js`	+1 attempt; locks 15 min at 5; resets count if expired lock
`resetLoginAttempts`	`models/User.js`	zeroes attempts, sets `lastLogin`, clears lock
`isLocked` (virtual)	`models/User.js`	lock-active check
`isValid` / `revoke`	`models/RefreshToken.js`	!revoked && !expired / set revoked+save
`getInstance` / `updateSingleton`	`models/ClinicInfo.js`	Singleton read (auto-create) / allowlisted assign+save
`validateFile`	`models/MediaAsset.js`	Type+size check vs `ALLOWED_TYPES`
`isInUse`	`models/MediaAsset.js`	`usedIn.length > 0`

Function	File	What
`getPublished` / `getBySlug` / `getAllForAdmin` / `stampPublishedAt`	`models/Treatment.js`	Public paginated+populated lean list w/ `$text` search (excludes `createdBy`) / published-by-slug / admin list w/ regex / publish timestamp helper
`getPublished` / `getBySlug` / `getAllForAdmin` / `stampPublishedAt`	`models/Article.js`	Same trio (list omits `body`, lean reads) + publish timestamp helper
`getLeads` / `updateLead` / `exportCursor`	`models/Lead.js`	Filtered+paginated list (populates treatment; sanitized-empty search ignored) / whitelisted update / lean streaming cursor
`getActive` / `getOrCreate`	`models/BookingProviderConfig.js`	Active singleton / auto-create singleton
`getSingleton` / `updateSingleton`	`models/Profile.js`	Singleton read (auto-create, populates photo/video) / whitelisted update (incl. `locationId`)
`getPublished` / `getAllForAdmin` (+ `stampPublishedAt`)	`models/Testimonial.js`, `models/GalleryItem.js`	Public filtered lean list / admin list; populate images/treatment; no consent, no review fields
`getPublished` / `getAllForAdmin`	`models/Video.js`, `models/CuratedPost.js`	Public filtered lean list / admin list

(+ `HomepageConfig.getSingleton/updateSingleton`, `RedirectMap.findByFromPath/getAllForAdmin`, `AnalyticsEvent.logEvent/getMonthlyReport`, `Location` (schema only, no statics) — same pattern as above.)

Services

Function	File	What
`generateTokens`	`authService.js`	Signs access+refresh JWTs (fresh `jti` each)
`hashToken`	`authService.js`	SHA-256 hex
`verifyAccessToken` / `verifyRefreshToken`	`authService.js`	`jwt.verify` with respective secret, null on any error
`login`	`authService.js`	Full credential check + lockout + token issue (uniform `Invalid credentials` for unknown/bad/disabled)
`refreshAccessToken`	`authService.js`	Validates stored hash, checks user active, rotates pair
`logout`	`authService.js`	Marks hash revoked (silent no-op if undecodable)
`getUserById`	`authService.js`	`User.findById`
`getClinicInfo` / `getClinicInfoAdmin` / `updateClinicInfo`	`clinicInfoService.js`	Singleton reads / whitelisted-field update via `updateSingleton`
`createLead`	`leadService.js`	Validates treatment id, inserts lead
`getLeads` / `updateLead` / `getLeadStats`	`leadService.js`	Passthroughs + status/source/date aggregations (total + last-7-days)
`initCloudinary` / `generatePublicId` / `buildVariantUrls`	`mediaService.js`	SDK config from env / `dr-ekta/<date>/<rand>.<ext>` id / deterministic `f_auto,q_auto` + 400/800/1600px delivery URLs
`validateFile`	`mediaService.js`	Passthrough to `MediaAsset.validateFile`
`generateSignedUrlCloudinary` / `generateSignedUrl`	`mediaService.js`	Validates, signs Cloudinary params incl. eager 400/800/1600px transforms (images), returns upload URL + public/optimized URLs + variants + signature bundle (h)
`confirmUpload`	`mediaService.js`	Re-validates type/size + requires non-empty `publicId` and http(s) `url`, creates MediaAsset (`storageKey =`

Function	File	What
		`publicId`) — controller checks `key` (send both)
`deleteMedia` / `listMedia`	`mediaService.js`	Guard-then-delete (Cloudinary destroy best-effort) / paginated list (optional `uploadedBy`, `mimeType`)
`generateSitemap`	`seoService.js`	XML of static routes + published treatments/articles/videos (videos by `_id`; entities escaped)
`generateRobotsTxt`	`seoService.js`	Allow `/`, disallow `/api/`+`/admin/`, sitemap line
`getRedirect`	`seoService.js`	`RedirectMap.findByFromPat` passthrough
`escapeXml`	`seoService.js`	XML entity escaper (`&<>"'`)

Controllers (handler → route)

- `authController`: `login` (→ `POST /auth/login`), `refresh`, `logout` (revoke-if-present + clear cookie), `me` (requires token, returns `req.user` fields).
- `clinicInfoController`: `getPublicClinicInfo` (+1 h cache headers + ETag), `getAdminClinicInfo`, `updateClinicInfo` (validated chain).
- `mediaController`: `getUploadUrl`, `confirmUpload` (400 on missing `key/url/…`, 422 on bad reference), `deleteMedia` (404; 409-if-in-use never fires), `listMedia`, `getMedia`.
- `articleController` / `treatmentController`: public list/detail + admin CRUD with direct `status`; shared `asyncHandler`; `NotFoundError` (404) on unknown ids.
- `testimonialController` / `galleryController`: public list + admin CRUD with direct `status`; content sanitized; no consent, no workflow.
- `videoController`: public list + admin CRUD (no workflow); update uses the create-shaped `video` validator — send the full object on video edits (partial bodies can 422).
- `curatedPostController`: public list + admin CRUD.
- `announcementController`: public ordered list + admin CRUD with direct `status`; body sanitized; `NotFoundError` 404s.
- `homepageController`: `getHomepage` (featured-first aggregate, caps 6/4/4/3), `getHomepageConfig`, `updateHomepageConfig` (unvalidated body → model whitelist).
- `leadController`: `getLeads`, `getLeadStats`, `exportLeads` (streamed CSV with formula escaping), `getLead`, `updateLead`, `deleteLead` (`NotFoundError` 404s on unknown id).
- `enquiryController`: `submitEnquiry` (honeypot → fake 200; captcha verified server-side when `RECAPTCHA_SECRET` set, else `invalid-token` → 422; medical-field rejection; treatment-exists check; lead insert; `enquiry_submit` analytics event; redacted `setImmediate` log), `verifyCaptcha` (5 s timeout, fail-closed when configured), `notifyNewEnquiry` (stub).
- `practoController`: `getBookingCta` (404 when inactive), `logPractoClick` (never-fails wrapper; also logs `practo_click` analytics event), `getBookingProvider`, `updateBookingProvider` (get-or-create), `getPractoClicks` (total + 30-day daily).

Note: these are bare async fns (no `asyncHandler`) — a thrown error outside the try/catch becomes an unhandled rejection.

- `profileController`: `getPublicProfile`, `getAdminProfile` (split routers), `updateProfile` (validated, bio sanitized, returns populated doc).
- `redirectController`: `getRedirects`, `getRedirect` (404), `createRedirect` (409 on duplicate `fromPath`), `updateRedirect` (`toPath`/`statusCode` only), `deleteRedirect`.
- `seoController`: `getSitemap` (XML), `getRobotsTxt` (text), `handleRedirect` (301/302 or 404).
- `analyticsController`: `getMonthlyReport` (query → `getMonthlyReport`), `getEvents` (manual filter + `{items,pagination}`, limit capped).

9. Standard Conventions

Response shapes - Success: the resource or `{ items, pagination }` or `{ message }` — no envelope. `pagination = { page, limit, total, pages }` (numbers; parsed with `parseInt`, so send digits). - Errors: `{ error: "<message>", code: "<CODE>" }`. Validation adds `details: [{ field, message, value }]` (only outside production). In `test` env, `stack`/`isOperational` are also attached. - Common codes: `VALIDATION_ERROR` (422), `AUTHENTICATION_ERROR` (401), `NOT_FOUND` (404), `CONFLICT_ERROR`/`DUPLICATE_REDIRECT` (409), `RATE_LIMIT_ERROR` (429), `INTERNAL_ERROR` (500), route-miss `NOT_FOUND` with `error: "Route not found"`.

Status codes you will see: `200` reads/writes, `201` creates (+ enquiry), `301/302` redirect resolver, `400` (confirm-missing-fields), `401`, `404`, `409`, `422`, `429`, `500`.

Pagination/filtering/sorting: `?page=&limit=` (defaults 1/20; analytics events default 50; `limit` capped at 100 on all list routes). Filters are exact-match (`status`, `category`, `source`, `treatmentTag`, `mimeType`) except `search` (`$text` on treatments/articles; regex on admin treatment/article/lead/redirect lists; sanitized-empty search ignored). Sorts are fixed per endpoint (newest first; videos `order` first; homepage featured first).

Dates: ISO-8601 strings (`2026-09-21T…Z`). Send `appointmentDate`-style fields as ISO strings; times as `"HH:mm"` (`10:00`); date-range filters as `?startDate=&endDate=` (anything `new Date()` parses).

File uploads: never POST files to the API. Flow: `POST /api/admin/media/upload` → browser POSTs multipart to Cloudinary `uploadUrl` → `POST /api/admin/media/confirm` with metadata (remember `key` + `publicId` with the same value, plus a valid http(s) `url`). Caps: images 5 MB (gif 3 MB), video 50 MB, pdf 10 MB, vtt 1 MB. JSON bodies are capped at **1 mb**.

Rate limits (per IP+path): public reads 100/min, enquiry 20/min, auth login/refresh 10/15 min. `429 { error, retryAfter }`. Disabled in tests unless `x-enable-rate-limit: true` is sent. In-process MemoryStore: limits are per-instance (fine for single-instance deploys).

CORS: allowlist from `FRONTEND_URL` (comma-separated), `credentials: true`, methods GET/POST/PUT/PATCH/DELETE/OPTIONS, headers `Content-Type, Authorization`. **Cookies cross-site:** the httpOnly refresh cookie is `SameSite=Lax` — fine for same-site admin+API, needs `Secure` + same-site context; prefer the Authorization header + body-token flow from a different origin.

Caching: public profile/clinic-info `max-age=3600`; treatments/articles/videos lists `max-age=300`; homepage `max-age=60`; sitemap/robots 1 h; everything else `no-store`; all admin `no-store`. After admin edits, bust the CDN/client cache for that path if one sits in front.

---

10. Frontend Integration Guide

Base URL + token setup

```
// config.js
export const API_BASE = process.env.REACT_APP_API_BASE ||
         'http://localhost:4000/api';
export const PUBLIC = `${API_BASE}/public`;
export const ADMIN = `${API_BASE}/admin`;
```

Reusable API client (axios) with refresh + retry

```
import axios from 'axios';
import { ADMIN } from './config';

let accessToken = null;          // keep in memory (not localStorage, if you
         can avoid it)
let refreshToken = localStorage.getItem('rt') || null;

export const api = axios.create({ baseURL: ADMIN, withCredentials: true });
api.interceptors.request.use(cfg => {
  if (accessToken) cfg.headers.Authorization = `Bearer ${accessToken}`;
  return cfg;
});

let refreshing = null;
api.interceptors.response.use(
  r => r,
  async err => {
    const orig = err.config;
    if (err.response?.status === 401 && !orig._retry) {
      orig._retry = true;
      refreshing ||= axios.post(`${ADMIN}/auth/refresh`, { refreshToken }, {
         withCredentials: true })
        .then(({ data }) => {
          accessToken = data.accessToken;
          refreshToken = data.refreshToken;
          localStorage.setItem('rt', refreshToken);
        }).finally(() => (refreshing = null));
      try { await refreshing; return api(orig); }
      catch { window.location.href = '/admin/login'; }
    }
    // Surface backend messages: err.response.data = { error, code, details?
         }
    return Promise.reject(err);
  }
);
```

```
export async function adminLogin(email, password) {
  const { data } = await axios.post(`${ADMIN}/auth/login`, { email, password
        }, { withCredentials: true });
  accessToken = data.accessToken; refreshToken = data.refreshToken;
  localStorage.setItem('rt', refreshToken);
  return data.user;
}
export function adminLogout() {
  axios.post(`${ADMIN}/auth/logout`, { refreshToken }).catch(() => {});
  accessToken = refreshToken = null; localStorage.removeItem('rt');
}
```

Page → endpoint mapping (public site)

Page / feature	Endpoints
Header/footer/contact	`GET /api/public/clinic-info`
Homepage	`GET /api/public/home/` (one call)
Treatment list / detail	`GET /api/public/treatments` → `GET /api/public/treatments/:slug`
Blog list / detail	`GET /api/public/articles` → `GET /api/public/articles/:slug` (detail has full `body`)
About doctor	`GET /api/public/profile/`
Testimonials section	`GET /api/public/testimonials/` (+ `treatmentTag` filter)
Smile gallery	`GET /api/public/gallery/` (before/after slider)
Videos	`GET /api/public/videos/` (+ `category`)
Instagram strip	`GET /api/public/instagram-posts/` (link `permalink`)
Enquiry form	`POST /api/public/enquiry` (hidden honeypot field; no medical fields)
Book button	`GET /api/public/booking-cta` → `bookingUrl`; fire `POST /api/public/leads/practo-click` on click (no await)

Admin panel mapping: login (`auth/login`), dashboard (`leads/stats`, `analytics/reports/monthly`), CMS lists with Draft/Published toggles (treatments/articles/testimonials/gallery/videos/instagram-posts), media library (`media/*` two-step upload), leads table + CSV (`leads/*`), clinic settings (`clinic-info`, `booking-provider`), SEO (`redirects/*`), homepage pinning (`home/config`).

Key user flows (exact call order) 1. *Visitor enquires:* `GET booking-cta` (render Book link) → fill form → `POST /enquiry {name, phone, treatmentInterest?, honeypot:""}` → show `message`; optionally `POST leads/practo-click` on book-click. 2. *Admin session:* `POST auth/login` → store pair → `GET clinic-info` (verify) → work with `Authorization` header → silent `POST auth/refresh` on 401 → `POST auth/logout` on exit. 3. *Publish a treatment:* `POST treatments` (draft, or `status: "published"` directly)

→ `PUT treatments/:id` edits incl. status toggle → visible on public list/detail when `published`. 4. *Publish a testimonial:* `POST testimonials {patientDisplayName, content, treatmentTag?}` (draft, or `status: "published"` directly) → `PUT testimonials/:id` edits incl. status toggle. No consent fields exist. 5. *Media attach:* `POST media/upload {filename,mimeType,size}` → browser-POST file to `uploadUrl` → `POST media/confirm {…, key, publicId, url}` → use returned `_id` in content forms.

Real-time: none (no websockets/SSE). Poll `leads` lists for freshness in the admin panel.

---

11. Known Gaps & Warnings

Resolved by the review (keep for history): `auth/me` protection (FIXED), admin-list shadowing (FIXED via split routers), not-found 500s (FIXED → 404s), media `integrations.config` crash (FIXED), sitemap `/videos/undefined` + no-op `escapeXml` (FIXED), unwired analytics events (FIXED — enquiry/click events logged), unwired `sanitizeRichText` (FIXED — called on all rich-text writes), public internal-field leaks (FIXED — `createdBy` excluded), captcha stub (FIXED when `RECAPTCHA_SECRET` set), in-memory CSV export (FIXED — cursor stream + escaping).

Still open with the backend owner (ordered by frontend impact):

1. **Media "in use" guard is dead** — nothing populates `usedIn`, so `DELETE /media/:id` never 409s. Deleting an image can break live pages referencing its URL. Confirm before adding delete buttons.
2. **Video update validator is create-shaped** — `PUT /videos/:id` runs the full `video` chain (`url/title/category` required-shaped), so partial updates can 422. Send the full object on video edits.
3. **Notifications are best-effort** — enquiries create the lead + analytics event but notify nobody (redacted console log only). Wire `NOTIFICATION_EMAIL`/SMTP or a queue before launch, or leads will sit unread.
4. **Dead config** — `.env:STORAGE_PROVIDER/AWS_*`, `PRACTO_*`/`GA_*`/`SMTP_*` envs are unreferenced (no sending/linking code reads them). Unused npm deps (`multer`, `csv-writer`, `node-cron`, `memorystore`, `etag`, `uuid`) were pruned from `package.json` + lockfile (review); `npm audit` is clean (0 vulnerabilities). `Location` has a schema but **zero endpoints and zero statics** — ask whether location filtering/switching is in scope.
5. **Small gotchas** — `device` on RefreshToken is never written (only `ip` is stored; harmless); homepage `featuredArticles` populate `heroImage`, which Article doesn't have (no-op); `practoController` handlers lack `asyncHandler` (a throw outside try/catch = unhandled rejection); `refModel: "Appointment"` has no matching model; `phone_click` source is never written by any code.

---

12. One-Day Integration Checklist

Morning — read-only + auth + public site: - [ ] Boot backend (`npm install`, `.env`, `npm run dev`) and hit `GET /api/public/health` + `GET /`. - [ ] Seed one admin user directly in Mongo (`users` collection; set `passwordHash` via bcrypt or ask owner for credentials) — there is no signup endpoint. - [ ] Implement the axios client above (base URLs, token attach, refresh-and-retry, `{error, code, details}` surfacing). - [ ] Build admin login/logout screens (`auth/login`, `auth/logout`); verify with `GET /api/admin/clinic-`

`info`. - [ ] Wire header/footer/contact (`clinic-info`), homepage (`home/`), treatment list+detail, article list+detail. - [ ] Wire about (`profile/`), testimonials, gallery, videos, Instagram strip (one list call each; respect `treatmentTag`/`category` filters).

Afternoon — forms, booking, admin panel: - [ ] Build enquiry form (`POST /enquiry` with honeypot, phone regex, no medical fields) + success/error states (422 details → field errors, 429 → cooldown message). - [ ] Wire Book button (`booking-cta` with 404 fallback; fire-and-forget `practo-click`). - [ ] Build admin CMS lists + create/edit forms for treatments/articles/testimonials/gallery (Draft/Published toggle, no workflow buttons). - [ ] Build media library (list, two-step upload, delete with “may break live pages” warning), leads table + `/stats` cards + CSV export (blob download), clinic/booking settings, redirects table, homepage pinning (`home/config`). - [ ] Add sitemap/robots link tags, empty-state handling for every list, and a global 401→login redirect. - [ ] Run through Section 11 with the backend owner; do not ship admin delete flows without their sign-off.